

Implementation of Data Augmentation using Affine Transformation

Project Title

Implementation of Data Augmentation using Affine Transformation

Objective

The objective of this project is to develop a generalized Python program that performs **image data augmentation using affine transformations**.

The program applies different transformations such as **translation, rotation, scaling, and shearing** to an input image and generates multiple augmented images.

The implementation uses homogeneous coordinates, transformation matrices, multiple transformations, image display, and saving of the generated images.

Introduction

Data augmentation is a technique used in machine learning and computer vision to increase the diversity of available training data.

Instead of collecting a completely new image for every possible variation, an existing image can be transformed to create additional training examples.

For example, an image can be:

- Shifted horizontally or vertically.
- Rotated.
- Enlarged or reduced.
- Slanted using shearing.

These transformations create variations while preserving much of the original image's information.

Affine transformations provide a mathematical framework for performing these geometric changes.

Algorithm

1. Read the input image.
2. Convert the image into an appropriate array representation.
3. Define transformation parameters.
4. Construct the translation matrix.
5. Construct the rotation matrix.
6. Construct the scaling matrix.
7. Construct the shearing matrix.
8. Represent image coordinates using homogeneous coordinates.
9. Apply the required transformation.

10. Generate the transformed image.
11. Repeat for different transformations.
12. Combine selected transformations using matrix multiplication.
13. Save the generated images.
14. Display the original and augmented images.
15. Compare the transformations.

Source code

```
import math
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# =====
# FUNCTION TO DISPLAY TRANSFORMATION MATRIX
# =====

def print_matrix(matrix, title):

    print("\n" + title + ":")

    for row in matrix:
        for value in row:
            print(f"{value:8.2f}", end=" ")
        print()

# =====
# TRANSLATION MATRIX
# =====

def translation_matrix(tx, ty):

    matrix = np.array([
        [1, 0, tx],
        [0, 1, ty],
        [0, 0, 1]
    ])

    return matrix

# =====
# ROTATION MATRIX
# =====

def rotation_matrix(angle):

    # Convert degrees to radians
    radians = math.radians(angle)
```

```

cos_value = math.cos(radians)
sin_value = math.sin(radians)

matrix = np.array([
    [cos_value, -sin_value, 0],
    [sin_value,  cos_value, 0],
    [0,          0,        1]
])

return matrix

# =====
# SCALING MATRIX
# =====

def scaling_matrix(sx, sy):

    matrix = np.array([
        [sx, 0, 0],
        [0, sy, 0],
        [0, 0, 1]
    ])

    return matrix

# =====
# SHEARING MATRIX
# =====

def shearing_matrix(shx, shy):

    matrix = np.array([
        [1, shx, 0],
        [shy, 1, 0],
        [0, 0, 1]
    ])

    return matrix

# =====
# APPLY AFFINE TRANSFORMATION
# =====

def apply_transformation(image, matrix):

    # Convert image to NumPy array
    image_array = np.array(image)

    height = image_array.shape[0]
    width = image_array.shape[1]

```

```
# Create empty output image
output = np.zeros_like(image_array)

# Find center of image
center_x = width / 2
center_y = height / 2

# Matrix to move center to origin
move_to_origin = np.array([
    [1, 0, -center_x],
    [0, 1, -center_y],
    [0, 0, 1]
])

# Matrix to move origin back to center
move_back = np.array([
    [1, 0, center_x],
    [0, 1, center_y],
    [0, 0, 1]
])

# Create transformation around image center
final_matrix = np.dot(
    move_back,
    np.dot(matrix, move_to_origin)
)

# Find inverse matrix
inverse_matrix = np.linalg.inv(final_matrix)

# Go through every pixel
for y in range(height):
    for x in range(width):
        # Homogeneous coordinate
        point = np.array([
            x,
            y,
            1
        ])

        # Find corresponding original point
        original_point = np.dot(
            inverse_matrix,
            point
        )

        original_x = int(round(original_point[0]))
        original_y = int(round(original_point[1]))

        # Check if point is inside image
        if (
```

```
        original_x >= 0
        and original_x < width
        and original_y >= 0
        and original_y < height
    ):

        output[y][x] = image_array[
            original_y
            ][original_x]

    return Image.fromarray(output)

# =====
# LOAD IMAGE
# =====

image_path = "input.jpg"

try:

    image = Image.open(image_path)

    # Convert to RGB
    image = image.convert("RGB")

except FileNotFoundError:

    print("Error: Image file not found.")
    print("Make sure input.jpg is in the same folder.")
    exit()

print("=" * 60)
print("DATA AUGMENTATION USING AFFINE TRANSFORMATION")
print("=" * 60)

# =====
# USER PARAMETERS
# =====

tx = 50
ty = 30

angle = 30

sx = 1.5
sy = 1.5

shx = 0.3
shy = 0.0
```

```
# =====
# CREATE TRANSFORMATION MATRICES
# =====

translation = translation_matrix(tx, ty)

rotation = rotation_matrix(angle)

scaling = scaling_matrix(sx, sy)

shearing = shearing_matrix(shx, shy)

# =====
# DISPLAY MATRICES
# =====

print_matrix(
    translation,
    "Translation Matrix"
)

print_matrix(
    rotation,
    "Rotation Matrix"
)

print_matrix(
    scaling,
    "Scaling Matrix"
)

print_matrix(
    shearing,
    "Shearing Matrix"
)

# =====
# APPLY TRANSFORMATIONS
# =====

translated_image = apply_transformation(
    image,
    translation
)

rotated_image = apply_transformation(
    image,
    rotation
)

scaled_image = apply_transformation(
    image,
```

```
        scaling
    )

    sheared_image = apply_transformation(
        image,
        shearing
    )

    # =====
    # COMBINE TRANSFORMATIONS
    # =====

    combined_matrix = np.dot(
        translation,
        rotation
    )

    combined_matrix = np.dot(
        combined_matrix,
        scaling
    )

    print_matrix(
        combined_matrix,
        "Combined Transformation Matrix"
    )

    combined_image = apply_transformation(
        image,
        combined_matrix
    )

    # =====
    # SAVE AUGMENTED IMAGES
    # =====

    translated_image.save("translated.jpg")

    rotated_image.save("rotated.jpg")

    scaled_image.save("scaled.jpg")

    sheared_image.save("sheared.jpg")

    combined_image.save("combined.jpg")

    print("\nAugmented images saved successfully.")
```

```
# =====  
# DISPLAY IMAGES  
# =====  
  
plt.figure(figsize=(12, 8))  
  
# Original image  
plt.subplot(2, 3, 1)  
plt.imshow(image)  
plt.title("Original Image")  
plt.axis("off")  
  
# Translated image  
plt.subplot(2, 3, 2)  
plt.imshow(translated_image)  
plt.title("Translated Image")  
plt.axis("off")  
  
# Rotated image  
plt.subplot(2, 3, 3)  
plt.imshow(rotated_image)  
plt.title("Rotated Image")  
plt.axis("off")  
  
# Scaled image  
plt.subplot(2, 3, 4)  
plt.imshow(scaled_image)  
plt.title("Scaled Image")  
plt.axis("off")  
  
# Sheared image  
plt.subplot(2, 3, 5)  
plt.imshow(sheared_image)  
plt.title("Sheared Image")  
plt.axis("off")  
  
# Combined image  
plt.subplot(2, 3, 6)  
plt.imshow(combined_image)  
plt.title("Combined Transformation")  
plt.axis("off")  
  
plt.tight_layout()  
  
plt.show()
```

Affine Transformation

An affine transformation can be represented using a matrix.

For a two-dimensional point:

$$(x, y)$$

we use **homogeneous coordinates**:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This allows translation, rotation, scaling, and shearing to be represented using 3×3 matrices.

A general affine transformation can be represented as:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Coordinates

Normally, a 2D point is represented as:

$$(x, y)$$

Using homogeneous coordinates, it becomes:

$$(x, y, 1)$$

or:

$$P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This representation makes it possible to express translation along with other transformations using matrix multiplication.

Translation

Translation moves an image from one position to another.

If the translation amounts are t_x and t_y , the transformation matrix is:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Therefore:

$$x' = x + t_x$$

$$y' = y + t_y$$

In the program, the user can modify `tx` and `ty` to control the amount of translation.

Rotation

Rotation changes the orientation of the image.

For an angle θ , the rotation matrix is:

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The Python program converts the angle from degrees to radians before calculating sine and cosine.

For example, for:

$$\theta = 30^\circ$$

the corresponding rotation matrix is approximately:

$$R = \begin{bmatrix} 0.866 & -0.5 & 0 \\ 0.5 & 0.866 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling

Scaling changes the size of the image.

The scaling matrix is:

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where:

- s_x is the horizontal scaling factor.
- s_y is the vertical scaling factor.

For example:

$$s_x = s_y = 1.2$$

increases the size of the image by approximately 20% in both directions.

Shearing

Shearing slants the image along one or both axes.

The shearing matrix used in the project is:

$$H = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where:

- sh_x controls horizontal shearing.
- sh_y controls vertical shearing.

For example:

$$sh_x = 0.2, \quad sh_y = 0$$

produces horizontal shearing.

Transformation About the Image Center

In the implementation, transformations such as rotation, scaling, and shearing can be performed around the **center of the image**.

If the image has width W and height H , its center is:

$$\left(\frac{W}{2}, \frac{H}{2} \right)$$

The transformation can be represented as:

$$M_{final} = T_{back} M T_{origin}$$

where T_{origin} moves the image center to the origin and T_{back} moves it back.

This prevents rotation and scaling from being unnecessarily anchored at the top-left corner.

Inverse Mapping

The program uses inverse mapping when assigning pixels to the output image.

For every pixel in the output image, its corresponding position in the original image is calculated using the inverse transformation matrix:

$$P_{original} = M^{-1} P_{output}$$

If the calculated point lies inside the original image boundaries, its pixel value is copied to the output image.

This approach helps avoid gaps between transformed pixels.

Software and Libraries Used

Python

Used for implementing the complete program.

NumPy

Used for:

- Matrix representation.
- Matrix multiplication.
- Matrix inversion.
- Homogeneous coordinate calculations.

Pillow

Used for:

- Reading the input image.
- Converting image data.
- Saving transformed images.

Matplotlib

Used to display the original and augmented images together for comparison.

Program Structure

Function	Purpose
<code>print_matrix()</code>	Displays transformation matrices
<code>translation_matrix()</code>	Creates translation matrix
<code>rotation_matrix()</code>	Creates rotation matrix
<code>scaling_matrix()</code>	Creates scaling matrix
<code>shearing_matrix()</code>	Creates shearing matrix
<code>apply_transformation()</code>	Applies transformation to image
Main section	Loads image, applies transformations, saves and displays results

This modular structure makes it easy to change transformation parameters and reuse the functions.

Input Image

The program reads an image from:

```
input.jpg
```



The input image is placed in the same folder as the Python program.

For example:

```
Data_Augmentation/  
├──  
├── main.py  
└── input.jpg
```

The same input image is then used to generate multiple augmented versions.

Transformation Parameters

The program allows the transformation parameters to be modified.

For example:

```
tx = 50
ty = 30

angle = 30

sx = 1.2
sy = 1.2

shx = 0.2
shy = 0.0
```

These values determine the amount of translation, rotation, scaling, and shearing.

Transformation Matrices Used

Translation

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling

$$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shearing

$$\begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Combined Transformation

Multiple affine transformations can be combined using matrix multiplication.

For example:

$$M = T \cdot R \cdot S$$

where:

- T is translation.
- R is rotation.
- S is scaling.

The resulting matrix represents the combined transformation.

The program creates this combined matrix and applies it to the original image.

Generated Outputs

The program generates the following images:

```
input.jpg
|
├── translated.jpg
├── rotated.jpg
├── scaled.jpg
├── sheared.jpg
└── combined.jpg
```

The implementation demonstrates four individual transformations plus a combined transformation.

Expected Output

The program displays:

- Original Image
- Translated Image
- Rotated Image
- Scaled Image
- Sheared Image
- Combined Transformation

These outputs allow the effect of each transformation to be visually compared.

Results

The program successfully generates different augmented versions of the input image.

```
=====
DATA AUGMENTATION USING AFFINE TRANSFORMATION
=====

Translation Matrix:
    1.00    0.00   50.00
    0.00    1.00   30.00
    0.00    0.00    1.00

Rotation Matrix:
    0.87   -0.50    0.00
    0.50    0.87    0.00
    0.00    0.00    1.00

Scaling Matrix:
    1.50    0.00    0.00
    0.00    1.50    0.00
    0.00    0.00    1.00

Shearing Matrix:
    1.00    0.30    0.00
    0.00    1.00    0.00
    0.00    0.00    1.00

Combined Transformation Matrix:
    1.30   -0.75   50.00
    0.75    1.30   30.00
    0.00    0.00    1.00

Augmented images saved successfully.
PS C:\Users\user\OneDrive\Desktop\GitHub\MFML-Assignment2> |
```

Translation

The image is shifted horizontally and vertically while maintaining its original shape.



Rotation

The image is rotated by the specified angle.



Scaling

The image is enlarged or reduced according to the specified scaling factors.



Shearing

The image is slanted horizontally or vertically.



Combined Transformation

Multiple transformations are applied sequentially through matrix multiplication, producing a more complex augmented image.



Observations

The following observations were made:

1. **Translation** changes the position of the image without changing its shape.
2. **Rotation** changes the orientation of the image.
3. **Scaling** changes the size of the image.
4. **Shearing** changes the geometric shape by slanting the image.
5. Combining transformations produces additional variations.
6. Homogeneous coordinates allow different geometric transformations to be represented using matrices.
7. Different transformation parameters produce different augmented versions of the same image.
8. These variations can increase the diversity of image data available for machine learning applications.

Applications of Data Augmentation

Affine-based image augmentation can be useful in:

- Image classification.
- Object recognition.
- Computer vision.
- Image detection.
- Pattern recognition.
- Machine learning model training.

For example, if a model is trained only on images in one orientation or position, it may perform poorly when the same object appears at a different position or orientation. Augmented images provide additional

variations during training.

Learning Outcomes

After completing this project, the following concepts were understood:

- Affine transformations.
- Homogeneous coordinates.
- Translation.
- Rotation.
- Scaling.
- Shearing.
- Matrix representation of geometric transformations.
- Combining transformations.
- Image processing using Python.
- Data augmentation in machine learning.
- Effects of geometric transformations on images.

Conclusion

The project successfully implements image data augmentation using affine transformations in Python.

The program reads an input image and applies translation, rotation, scaling, and shearing using their corresponding transformation matrices. Homogeneous coordinates are used to represent image points, while matrix multiplication is used to combine multiple transformations.

The generated images demonstrate how a single image can be transformed into multiple variations. Such transformations can increase the diversity of training data and are useful in machine learning and computer vision applications.

The implementation is modular and generalized, allowing different images and transformation parameters to be used without changing the core functions.